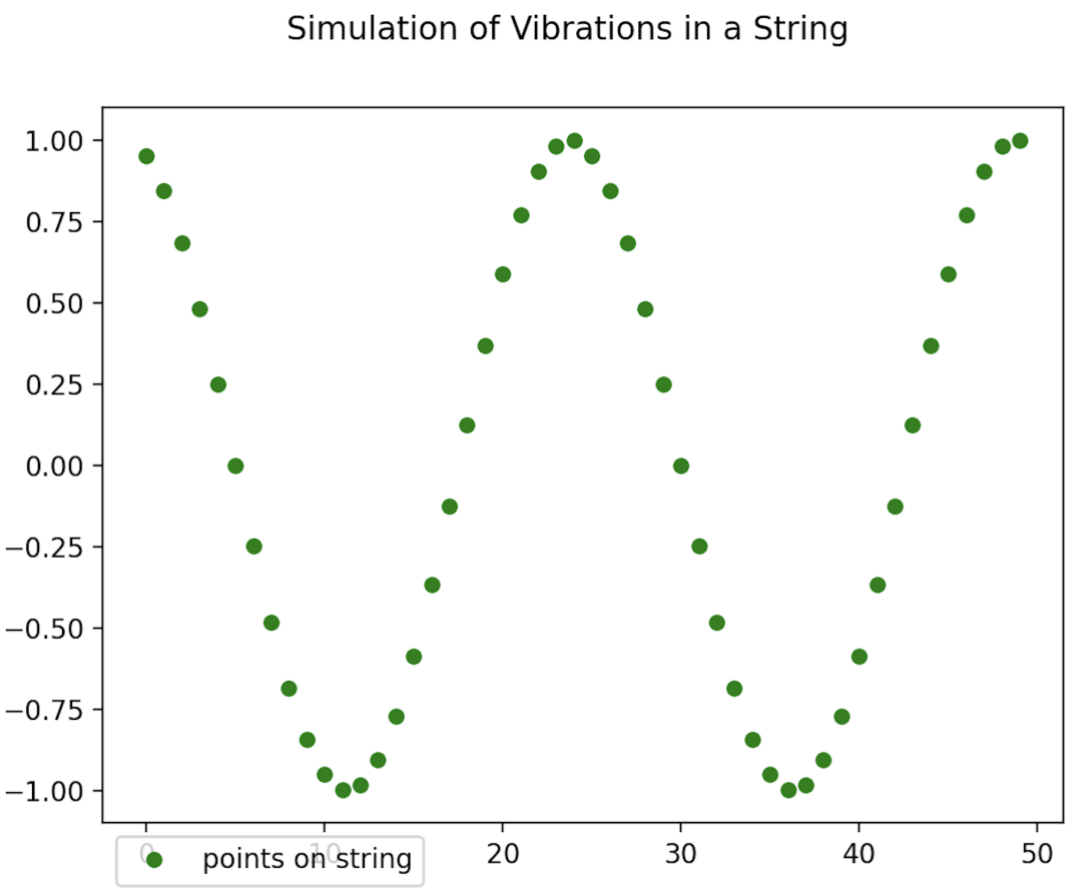
Part 1
Removing hard code form files

String.c programme
In this version of the program, hard-coded parameters were removed and replaced with command-line arguments to improve flexibility and usability. A structure was introduced to store multiple input values, including the number of points, number of cycles, sampling rate, and output file path. The check_args() function was modified to parse and validate these inputs, ensuring they are positive and correctly provided. This allows users to control the simulation setup and output location without modifying the source code. Additionally, error handling was added for memory allocation and file operations to improve robustness. Overall, these changes make the program more general, reusable, and user-friendly.

Inputs are :
```
../bin/string_wave 50 5 25
/Users/dhaakirahfasetire/Desktop/data/string_wave.csv
```

50 points
5 cycles
25 samples per cycle
output written to your Desktop data folder

Animate line file programme
Compared with the original script, this version removes the hard-coded output filename and lets the user specify both the input CSV file and the output GIF file from the command line. A new function was added to validate these arguments and make sure the input is a CSV file and the output is a GIF file. The rest of the programme is the same as provided.



Part 2

For the string.c program, the main iteration patterns are the loop over time steps and the loop over points along the string. The loop must remain sequential, because each new state of the string depends on the positions from the previous time step. There is also spatial iteration inside the update position function, where each point is updated based on neighbouring information. This part is the one that can be parallelised by dividing the string into chunks along its length. Each process can handle the points inside its own chunk independently.
If the string is split across several processes, the interior points of each chunk can be updated in parallel without any problem, but boundary points between chunks require communication. The last point of one chunk and the first point of the next chunk are neighbours, so processes must exchange these edge values after each time step. This can be dealt with using boundary communication between neighbouring processes.

The strategy for aggregation is to collect the results in memory at each time step and let only the root process write to the output file. This avoids file corruption and race conditions that would occur if multiple processes tried to write to the same file simultaneously.In the parallel version, each process would own a chunk of the string and update the points in that chunk. After each time step, the local position data from all processes can be sent to the root process using a collective communication routine. This in-memory aggregation strategy is preferable to writing separate partial files to disk and merging them later. Writing to disk from every process would introduce extra I/O overhead, make the workflow more complicated, and likely reduce performance.

A parallel MPI version of the string wave simulation was implemented by splitting the string evenly into chunks along its length, with each process updating one chunk. The time loop remained sequential, since each new time step depends on the previous one, but the spatial updates were carried out in parallel. Boundary values were exchanged between neighbouring processes using point-to-point communication so that the first point in each chunk could be updated correctly. After each time step, the local chunks were gathered on the root process using MPI_Gather(), and only the root process wrote the full string state to the output file. This avoided unsafe simultaneous file writing .

Benchmark time results

Points	Serial Avg Runtime (s)	MPI Avg Runtime (s)	Faster Version
50	0.0100	0.1333	Serial
500	0.0183	0.0873	Serial
5000	0.0787	0.1463	Serial

For all tested problem sizes, the serial version was faster than the MPI version. This means the task does not yet demand a parallel solution at these sizes. The overhead of MPI communication, boundary exchange, gathering the data to the root process, and root-only file writing is larger than the performance gain from splitting the work across processes. The gap becomes smaller as the number of points increases, which suggests that a parallel solution may become more viable for much larger problem sizes.

Part 3

String_wave_force.c
String-wave_....gif

Applying the spring force on each point.
To implement this a position and velocity array must be added. New postion and velocity arrays will be kept tracked off .The update position function was changed to reflect this.

The original rule was replaced with a simple spring–mass model, where each point on the string is treated as a mass connected to its neighbouring points by springs. In this implementation, each point has a position and velocity, and its motion is determined by the forces from adjacent points using Hooke's law.

The force on each interior point is calculated based on the difference between its displacement and that of its neighbours. This produces a more realistic propagation of motion along the string .

The first point on the string is driven by a sinusoidal function, which forces it to oscillate between -1 and 1. This acts as the input energy source for the system, causing waves to

propagate along the string. The final point is fixed, which leads to reflections of the wave back along the string.

To improve the behaviour of the simulation, the number of cycles was increased from 20 to 40. This allowed the animation to run for a longer time, ensuring that the wave propagates across the full length of the string instead of only a portion of it.

Additionally, the spring constant $k$was reduced to make the motion less abrupt. Lowering k decreases the strength of the restoring force between points, which results in smoother and less jumpy oscillations.

The resulting behaviour shows that the left side of the string responds first and with the largest amplitude, as it is directly driven by the external force. The disturbance then propagates along the string, gradually affecting points further to the right. Over time, the entire string begins to oscillate, with evidence of wave propagation and reflection due to the fixed boundary at the far end.

Step compile to generate animation

```
gcc hpqc_week5/string_wave_force.c -o ../bin/string_wave_force -lm

../bin/string_wave_force 50

python hpqc_week5/animate_line_file.py

/Users/dhaakirahfasetire/Desktop/data/string_wave_spring.csv

/Users/dhaakirahfasetire/Desktop/data/string_wave_spring.gif
```